06 DECORATORS IN NESTJS

A Decorator is special syntax — @SomethingLikeThis() placed above a class, method, property, or parameter — that attaches METADATA to it. Nest reads that metadata at startup to decide how to treat that class/method (register it as a route, mark it injectable, extract a value from the request, etc.).

Decorators don't run your business logic themselves — they just LABEL things so Nest's runtime knows what to do with them.

THE MAIN CATEGORIES

```
TYPE                  | EXAMPLES                                | APPLIED TO           | PURPOSE
----------------------|-----------------------------------------|----------------------|-------------
------------------------------
Class decorators      | @Module(), @Controller(), @Injectable() | Whole class          | Tells Nest
what role this class plays
Method decorators     | @Get(), @Post(), @Put(), @Delete()      | Controller methods   | Maps an
HTTP method+route to this method
Parameter decorators  | @Param(), @Body(), @Query(), @Headers() | Method arguments     | Extracts a
specific piece of the request
Other                 | @Inject(), @Optional()                  | Constructor params   | Fine-tunes
how DI resolves a dependency
```

EXAMPLE PUTTING SEVERAL TOGETHER

```ts
@Controller('cats')                // class decorator
export class CatsController {
  constructor(
    private readonly catsService: CatsService,   // no decorator needed
  ) {}                                            // Nest infers by type

  @Get(':id')                      // method decorator
  findOne(@Param('id') id: string) { // parameter decorator
    return this.catsService.findOne(id);
  }
}
```

- @Controller('cats') marks the class, sets base route "/cats".
- @Get(':id') marks this method as the handler for
- GET /cats/:id.
- @Param('id') pulls the ":id" segment out of the URL and passes it as the argument.

WALKTHROUGH OF THE MOST COMMON DECORATORS

```
@Module({...})
  Groups related controllers + providers into one unit. Every Nest app
  has at least a root module (AppModule).
```

@Controller('path') Marks a class as a controller and sets its base route prefix.

```
@Injectable()
  Marks a class as a Provider — something Nest's DI container can
  create and inject wherever it's needed (typically a service).

@Get() / @Post() / @Put() / @Patch() / @Delete()
  HTTP-method decorators on controller methods. Nest maps the
  request's method + path to exactly one of these handlers.
```

@Param('name') Extracts a value from the URL path, e.g. /cats/:id → @Param('id').

```
@Query('name')
  Extracts a value from the query string, e.g. ?breed=persian ->
  @Query('breed').

@Body()
  Extracts and parses the JSON request body, typically typed as a DTO
  class (e.g. @Body() dto: CreateCatDto).
```

@Headers('name') Extracts a specific request header (e.g. 'authorization').

@UseGuards(SomeGuard) Attaches a guard (implements CanActivate) to a controller or route — runs BEFORE the handler to decide if the request is allowed through (auth checks, role checks).

@UsePipes(SomePipe) Attaches a pipe (e.g. ValidationPipe) that transforms/validates incoming data before it reaches the handler.

```
@UseInterceptors(SomeInterceptor)
  Wraps the handler to run logic before AND after it (logging,
  response transformation, timeouts).
```

@Inject(TOKEN) Manually tells Nest which provider to inject when it can't be inferred purely from the TypeScript type (e.g. injecting a value by a custom token/string instead of a class).

WHY NEST LEANS SO HEAVILY ON DECORATORS

Under the hood, decorators attach metadata via the "reflect-metadata" library. Nest reads that metadata at boot time to auto-wire everything — routes, dependency injection, validation — without you writing manual registration code yourself.

This is the core difference from a minimal framework like Express: in Express you manually call app.get('/cats/:id', handler) and manually pull req.params.id yourself. In Nest, decorators declare the same intent declaratively, and the framework does the wiring for you at startup.

★ RULE OF THUMB

A decorator is metadata, not logic. If you're wondering "what does this decorator actually DO at runtime" — the answer is almost always "it marks this thing so Nest's startup process can find it and wire it up correctly." The real logic still lives in the method body underneath it.

ANGULAR vs NESTJS DECORATORS

This isn't a coincidence — NestJS was deliberately designed to FEEL like Angular, but on the backend. Same decorator-based, module-based architecture; same underlying TypeScript decorator mechanism (reflect-metadata); same "@Module ties everything together" idea. Frontend concepts map almost 1-to-1 onto backend equivalents.

SIDE-BY-SIDE MAPPING TABLE

```
CONCEPT              | ANGULAR                          | NESTJS
---------------------|----------------------------------|---------------------------------
Grouping unit        | @NgModule({...})                 | @Module({...})
Main building block  | @Component({...})                | @Controller('path')
Injectable class     | @Injectable()                    | @Injectable()
Injecting a dependency | constructor(private svc: Service) | constructor(private svc: Service)
Reading input        | @Input()                         | @Param() / @Query() / @Body()
Emitting output      | @Output()                        | return value / @Res()
Route definition     | RouterModule / [routerLink]      | @Get() / @Post() / @Put() / @Delete()
Guarding access      | CanActivate (route guards)       | @UseGuards() + CanActivate
Transform/validate data | Angular Forms / Validators    | @UsePipes() + ValidationPipe
Intercept lifecycle  | Angular Interceptors (HttpClient)| @UseInterceptors()
Custom decorator     | Attribute directives             | Custom param decorators
                     | (@Directive())                   | (createParamDecorator)
```

EXAMPLE 1 — MODULE

ANGULAR:

```
@NgModule({
  declarations: [CatsListComponent],
  imports: [CommonModule],
  providers: [CatsService],
})
export class CatsModule {}
```

NESTJS:

```
@Module({
  controllers: [CatsController],
  imports: [DatabaseModule],
  providers: [CatsService],
})
export class CatsModule {}
```

Both group related pieces together and declare what this "feature slice" needs (imports) and provides (providers).

EXAMPLE 2 — INJECTABLE SERVICE (near-identical)

ANGULAR:

```
@Injectable({ providedIn: 'root' })
export class CatsService {
  getCats() { return this.http.get('/api/cats'); }
}
```

NESTJS:

```
@Injectable()
export class CatsService {
  getCats() { return this.catsRepo.find(); }
}
```

Same decorator, same idea: "Let the DI container manage and hand out instances of this class." The only real difference is WHAT it talks to (an HTTP client fetching data in Angular vs. a database repository serving data in Nest) — because one runs in the browser, one runs on the server.

EXAMPLE 3 — RECEIVING INPUT

```
ANGULAR (component receives data from its parent template):
  @Component({ selector: 'cat-card' })
  export class CatCardComponent {
    @Input() catName: string;   // <cat-card [catName]="cat.name">
  }

NESTJS (controller receives data from an HTTP request):
  @Controller('cats')
  export class CatsController {
    @Get(':id')
    findOne(@Param('id') id: string) {   // GET /cats/5 -> id = '5'
      return this.catsService.findOne(id);
    }
  }
```

Both decorators answer the same question — "where does this value come from?" — just for a different kind of caller (a parent template vs. an HTTP client).

WHERE THE ANALOGY BREAKS DOWN

- Angular decorators mostly describe UI/rendering concerns (components, templates, change detection, inputs/outputs for data binding).
- Nest decorators mostly describe HTTP/server concerns (routes, request parsing, guards for auth, response shaping).
- Angular has no direct equivalent of @Get()/@Post() — routing in Angular is template/URL-driven (RouterModule), not method-decorator driven.
- Nest has no direct equivalent of @Input()/@Output() data-binding — Nest doesn't render UI, so there's no parent/child component relationship to bind across.

WHY THIS MATTERS IF YOU KNOW ANGULAR

If you've used Angular, NestJS should feel remarkably familiar:

- "Everything is a decorated class" → same.
- "Modules declare what's inside them and what they need" → same.
- "@Injectable() + constructor injection = DI" → same.
- The mental shift is just: Angular decorators describe UI/browser concerns, Nest decorators describe HTTP/server concerns — the PATTERN transfers, the DOMAIN doesn't.

See also: **theory/02-controller.txt**, **theory/03-service.txt**